23 REFLECTOR & METADATA — HOW @Roles() ACTUALLY REACHES RolesGuard

Reflector is a small NestJS utility class with one job: READ metadata that a decorator like @SetMetadata() previously ATTACHED to a class or method. It's the missing link between "a decorator was put on a route" and "a guard can see that and act on it."

```
@Roles(Role.Admin)          <-- WRITES metadata onto remove()
@UseGuards(AuthGuard, RolesGuard)
@Delete(':id')
remove(...) { ... }

this.reflector.get(ROLES_KEY, context.getHandler())
                            <-- READS that same metadata back,
                                inside RolesGuard.canActivate()
```

Without Reflector, RolesGuard would have no way to know WHICH roles a given route requires — @Roles(Role.Admin) on its own does nothing at runtime except stash a value; something has to go looking for it.

WHAT "METADATA" MEANS HERE

This has nothing to do with HTTP metadata or database metadata. It's TypeScript's reflect-metadata: every class and every method can carry an invisible key → value map attached to it, set at decoration time and read later via reflection. Nest wraps this low-level mechanism in two friendlier pieces:

@SetMetadata(key, value)	WRITES one entry onto whatever the decorator is stacked on (a class or a method).
Reflector	READS entries back out, given the same key and a reference to that class/method.

This project's actual roles.decorator.ts is a thin wrapper around @SetMetadata:

```
// src/guards/roles/roles.decorator.ts
export const ROLES_KEY = 'roles';
export const Roles = (...roles: Role[]) => SetMetadata(ROLES_KEY, roles);
```

Calling @Roles(Role.Admin) on the remove() handler is EXACTLY the same as writing @SetMetadata('roles', [Role.Admin]) directly — Roles(...) just gives it a name, a fixed key (ROLES_KEY), and a typed signature (Role[] instead of any[]).

WHERE THE METADATA LIVES: HANDLER vs CLASS

Metadata can be attached at two different levels, and Reflector needs to be told WHICH one to read from:

```
@Roles(Role.Admin)            // <-- attached to the METHOD
@Delete(':id')
remove(...) { ... }
```

context.getHandler()	returns a reference to THIS specific method (remove). Metadata set by a decorator stacked directly on remove() lives here.
context.getClass()	returns a reference to the whole TaskController class. Metadata set by a decorator stacked on the @Controller() itself (protecting EVERY route) lives here instead.

This project's roles.guard.ts reads only the handler level:

```
// src/guards/roles/roles.guard.ts
const requiredRoles = this.reflector.get<Role[]>(
  ROLES_KEY,
  context.getHandler(),   // <-- exact method being called
);
```

Using getClass() here by mistake would look for @Roles() on TaskController itself and find nothing (it's not there) — the guard would silently treat every route as unrestricted. Getting getHandler()/getClass() backwards is one of the most common RolesGuard bugs, precisely because both compile fine and only fail at runtime.

reflector.get() — READING ONE LEVEL

```
this.reflector.get<Role[]>(ROLES_KEY, context.getHandler())
```

ROLES_KEY	the same string key @SetMetadata wrote under ('roles') — must match exactly, which is WHY the project exports it as a shared constant instead of retyping 'roles' in two files (typo-proofing).
context.getHandler()	WHERE to look — just the method-level metadata, ignoring anything on the class.

```
Returns Role[] | undefined — undefined if @Roles() was never applied
to this route at all, which is exactly what roles.guard.ts checks
for:

    if (!requiredRoles || requiredRoles.length === 0) {
      return true; // route has no @Roles() -> no restriction
    }
```

reflector.getAllAndOverride() — READING BOTH LEVELS AT ONCE

A common real-world upgrade: let a controller set a DEFAULT that individual routes can override.

```
this.reflector.getAllAndOverride<Role[]>(ROLES_KEY, [
  context.getHandler(),   // checked FIRST
  context.getClass(),     // fallback if handler has none
]);
```

This checks the method first; if that method has no @Roles() of its own, it falls back to whatever's on the controller class. It's how Nest's own official RBAC example is written — one controller-level @Roles('user') as a baseline, overridden per-route where a stricter role is needed. This project's guard doesn't need that yet (only one route is protected), but it's the natural next step if more routes start using @Roles().

WHY Reflector NEEDS DEPENDENCY INJECTION

```
// src/guards/roles/roles.guard.ts
constructor(private readonly reflector: Reflector) {}
```

Reflector is injected, not imported as a static utility, because it's a real Nest provider — this is also WHY RolesGuard must be applied through @UseGuards() or APP_GUARD (both let Nest's DI container construct it) rather than `new RolesGuard()` by hand. A manually constructed instance has no reflector, and every reflector.get() call would throw or return undefined. Same DI requirement covered generally in theory/07-dependency-injection.txt.

THE FULL ROUND TRIP, TRACED

```
1. @Roles(Role.Admin) on remove()
     -> calls SetMetadata('roles', [Role.Admin])
     -> reflect-metadata stores { roles: [Role.Admin] } on the
        remove method's metadata map

2. Request hits DELETE /task/1
     -> AuthGuard runs first, validates x-api-key, sets request.user

3. RolesGuard.canActivate(context) runs
     -> context.getHandler() resolves to the remove method
     -> this.reflector.get('roles', <remove method>)
        walks reflect-metadata's storage and returns [Role.Admin]

4. Guard compares request.user.role against [Role.Admin]
     -> match  -> return true  -> handler runs
     -> no match -> throw ForbiddenException -> 403
```

★ RULE OF THUMB

Think of @SetMetadata (and anything built on it, like @Roles()) as LEAVING A NOTE on a route, and Reflector as the only thing that goes back and READS that note. The note by itself changes nothing — it's inert until some guard, interceptor, or pipe explicitly calls reflector.get(SAME_KEY, SAME_LEVEL) to look it up. Always match the key exactly (use a shared constant, never a repeated string literal) and always match the level (getHandler() for method decorators, getClass() for controller-level ones, or getAllAndOverride() for both).

See also: theory/22-role-based-authorization.txt, theory/17-guards.txt, theory/06-decorators.txt, theory/07-dependency-injection.txt </content>